

IDS Pipeline Runner Scripts Documentation

Purpose: These are master control scripts that orchestrate the entire IDS pipeline - they handle starting/stopping all components, checking status, and managing the complete workflow.

Overview

There are **two pipeline runner scripts**, one for each capture mode:

Script	Mode	Use Case
<code>run_afpacket_mode.sh</code>	AF_PACKET	USB adapters, testing, any interface
<code>run_dpdk_mode.sh</code>	DPDK	High-performance, Intel/Mellanox NICs

Both scripts provide:

- ✔ Interactive menu-driven interface
- ✔ Automated component startup/shutdown
- ✔ Status checking and health monitoring
- ✔ Log viewing
- ✔ Command-line operation (for automation)

Table of Contents

- [AF_PACKET Mode Runner \(`run\_afpacket\_mode.sh`\)](#)
- [DPDK Mode Runner \(`run\_dpdk\_mode.sh`\)](#)
- [Common Features](#)
- [Usage Examples](#)
- [Architecture Diagrams](#)

PROF

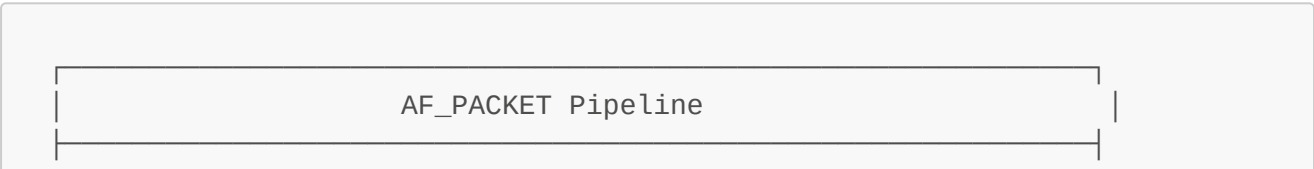
AF_PACKET Mode Runner

What It Does

Purpose: One-stop script to manage the complete IDS pipeline using AF_PACKET mode (standard Linux packet capture).

File: `run_afpacket_mode.sh`

Architecture It Manages



```

Network Interface (USB/Ethernet)
    ↓
Suricata (AF_PACKET mode) → eve.json file
    ↓
Kafka Bridge → Reads file & streams to Kafka
    ↓
Kafka Topic: suricata-alerts
    ↓
ML Consumer → Predictions
    ↓
Kafka Topic: ml-predictions

```

Key Functions Breakdown

1. Initialization & Checks

`print_header()`

- Displays colorful banner with mode information
- Shows that script is running in AF_PACKET mode
- Makes it clear this is USB-compatible

`check_root()`

- Verifies script is run with **sudo**
- **Why needed:** Interface configuration and Suricata require root

`load_config()`

- Loads settings from `dpgk_suricata_ml_pipeline/config/pipeline.conf`
- Reads variables like:
 - `NETWORK_INTERFACE` - Which interface to capture on
 - `KAFKA_BOOTSTRAP_SERVERS` - Kafka connection details
 - `SURICATA_LOG_DIR` - Where logs are stored

`check_dependencies()`

- Validates required software is installed:

- **Suricata** - IDS engine
- **Kafka** - Message broker
- **tcpreplay** - For PCAP replay
- **Python3** - For ML consumer
- Exits with error if any dependency missing

```
check_interface()
```

- Verifies network interface exists
- Checks if **NETWORK_INTERFACE** is configured
- Lists available interfaces if specified one not found
- Example check:

```
if ! ip link show "$NETWORK_INTERFACE" > /dev/null 2>&1; then
    echo "Interface not found!"
    ip link show # Show what's available
fi
```

2. Component Startup Functions

Each function starts one pipeline component and verifies it's running:

start_kafka()

What it does:

1. Checks if Kafka already running (**pgrep -f "kafka.Kafka"**)
2. If not running, calls **02_setup_kafka.sh**
3. Waits 3 seconds for startup
4. Verifies Kafka process exists
5. Exits with error if startup failed

Why 3 second wait: Kafka needs time to initialize Zookeeper connection

start_suricata()

What it does:

1. Checks if Suricata already running
2. Calls **03_start_suricata_afpacket.sh**
3. Starts Suricata with AF_PACKET capture on specified interface
4. Waits 3 seconds
5. Verifies Suricata process running with **--af-packet** flag

Process check:

```
if pgrep -f "suricata.*--af-packet" > /dev/null; then
    echo "✓ Suricata running"
fi
```

`start_kafka_bridge()`

What it does:

1. Checks if bridge script already running
2. Calls `04_start_kafka_bridge.sh`
3. Starts `suricata_kafka_bridge.py` in background
4. Bridge reads `eve.json` and streams to Kafka
5. Waits 2 seconds
6. Verifies bridge process exists

Why needed in AF_PACKET mode: Suricata writes to file, not directly to Kafka

`start_ml_consumer()`

What it does:

1. Checks if ML consumer already running
2. Calls `05_start_ml_consumer.sh` in background (&)
3. Starts `ml_kafka_consumer.py` daemon
4. Waits 3 seconds for initialization
5. Verifies ML consumer process running

Background execution (&): Allows script to continue while consumer runs

PROF

3. Status & Monitoring Functions

`show_status()`

What it does:

Displays comprehensive system status with color-coded indicators:

1. Kafka Status

```
if pgrep -f "kafka.Kafka" > /dev/null; then
    echo "✓ Kafka: Running"
else
    echo "✗ Kafka: Not running"
fi
```

2. Suricata Status

- Shows if running
- Displays PID
- Shows which interface it's capturing on

```
SURICATA_PID=$(pgrep -f "suricata.*--af-packet")
echo "PID: $SURICATA_PID"
echo "Interface: $NETWORK_INTERFACE"
```

3. Kafka Bridge Status

- Shows if bridge is streaming events

4. ML Consumer Status

- Shows if ML predictions are running

5. Network Interface Status

- Shows if interface is UP
- Checks if promiscuous mode enabled (required for packet capture)

```
if ip link show "$NETWORK_INTERFACE" | grep -q "PROMISC"; then
    echo "✓ Promiscuous mode enabled"
fi
```

Color codes:

-  Green ✓ = Running/OK
-  Red ✗ = Not running
-  Yellow ⚠ = Warning

PROF

view_logs()

What it does:

Interactive log viewer with 4 options:

1. Suricata logs - IDS engine logs

```
tail -f /var/log/suricata/suricata.log
```

2. ML consumer logs - ML prediction logs

```
tail -f dpdk_suricata_ml_pipeline/logs/ml/ml_consumer.log
```

3. Kafka bridge logs - File-to-Kafka streaming logs

```
tail -f dpdk_suricata_ml_pipeline/logs/kafka_bridge.log
```

4. All logs - Monitors all simultaneously

```
tail -f suricata.log ml_consumer.log kafka_bridge.log
```

Uses **tail -f**: Follows log files in real-time (like watching live feed)

4. Shutdown Functions

stop_all()

What it does:

Gracefully stops all components **in correct order** (reverse of startup):

Shutdown sequence:

- | | |
|-----------------|------------------|
| 1. ML Consumer | ← Top of chain |
| 2. Kafka Bridge | ← Middle |
| 3. Suricata | ← Source |
| 4. Kafka | ← Infrastructure |

PROF

Why this order?

- ML Consumer depends on Kafka events → stop first
- Bridge depends on Suricata logs → stop second
- Suricata generates events → stop third
- Kafka is infrastructure → stop last

Implementation:

```
# Stop ML consumer
if pgrep -f "ml_kafka_consumer.py" > /dev/null; then
    pkill -f "ml_kafka_consumer.py"
    echo "✓ ML consumer stopped"
fi
```

```
# Stop Kafka bridge
if pgrep -f "suricata_kafka_bridge.py" > /dev/null; then
    pkill -f "suricata_kafka_bridge.py"
    echo "✓ Kafka bridge stopped"
fi

# Stop Suricata
if pgrep -f "suricata" > /dev/null; then
    pkill -f "suricata"
    sleep 2 # Wait for clean shutdown
    echo "✓ Suricata stopped"
fi

# Stop Kafka (calls stop_all.sh script)
if pgrep -f "kafka.Kafka" > /dev/null; then
    bash "${PIPELINE_SCRIPTS}/stop_all.sh"
    echo "✓ Kafka stopped"
fi
```

sleep 2 after Suricata: Gives time for Suricata to flush buffers and close files properly

5. Additional Functions

setup_external_capture()

What it does:

1. Calls `00_setup_external_capture.sh`
2. Configures network interface for capture
3. Sets up isolated network (192.168.100.0/24)
4. Enables promiscuous mode
5. Configures firewall rules

PROF

When to use: First-time setup or after reboot

replay_traffic()

What it does:

1. Calls `05_replay_traffic.sh`
 2. Replays PCAP files to test IDS
 3. Useful for testing without real attacks
-

6. Interactive Menu System

show_menu()

Displays options:

```
===== MENU =====
1) Start Complete Pipeline (Kafka + Suricata + ML)
2) Start Kafka Only
3) Start Suricata Only (AF_PACKET)
4) Start ML Consumer Only
5) Start Kafka Bridge Only
6) Replay Traffic (PCAP)
7) Check Status
8) View Logs
9) Setup External Capture 
10) Stop All Services
0) Exit
=====
```

main() - Menu Loop

How it works:

```
while true; do
    show_menu
    read -p "Enter choice [0-10]: " choice

    case $choice in
        1) # Start complete pipeline
            start_kafka
            start_suricata
            start_kafka_bridge
            start_ml_consumer
            show_status
            ;;
        2) start_kafka ;;
        3) start_suricata ;;
        # ... etc
        0) exit 0 ;;
    esac

    read -p "Press Enter to continue..."
done
```

Menu loop benefits:

- User doesn't need to remember commands
- Can perform multiple operations without restarting
- Visual feedback after each action

Command-Line Usage (Non-Interactive)

Script supports direct execution for automation:

```
# Start complete pipeline
sudo ./run_afpacket_mode.sh start

# Individual components
sudo ./run_afpacket_mode.sh kafka      # Start Kafka only
sudo ./run_afpacket_mode.sh suricata   # Start Suricata only
sudo ./run_afpacket_mode.sh ml         # Start ML consumer only
sudo ./run_afpacket_mode.sh bridge     # Start Kafka bridge only

# Utilities
sudo ./run_afpacket_mode.sh status     # Check status
sudo ./run_afpacket_mode.sh logs       # View logs
sudo ./run_afpacket_mode.sh setup      # Setup external capture
sudo ./run_afpacket_mode.sh stop       # Stop everything
```

Implementation:

```
if [ $# -gt 0 ]; then # If argument provided
    case $1 in
        start|1)
            start_kafka
            start_suricata
            start_kafka_bridge
            start_ml_consumer
            exit 0
            ;;
        kafka|2) start_kafka; exit 0 ;;
        # ... etc
    esac
fi
```

PROF

Why useful:

- Automation scripts can call it
- Can integrate with systemd or cron
- Scripting workflows

Color Coding System

Makes output readable and visually clear:

```
RED='\033[0;31m'      # Errors
GREEN='\033[0;32m'     # Success
YELLOW='\033[1;33m'    # Warnings
```

```
BLUE='\033[0;34m'      # Info
CYAN='\033[0;36m'      # Headers
MAGENTA='\033[0;35m'   # Menus
BOLD='\033[1m'         # Emphasis
NC='\033[0m'           # Reset (No Color)
```

Example usage:

```
echo -e "${GREEN}✓ Service started${NC}"
echo -e "${RED}× Service failed${NC}"
echo -e "${YELLOW}⚠ Warning: Already running${NC}"
```

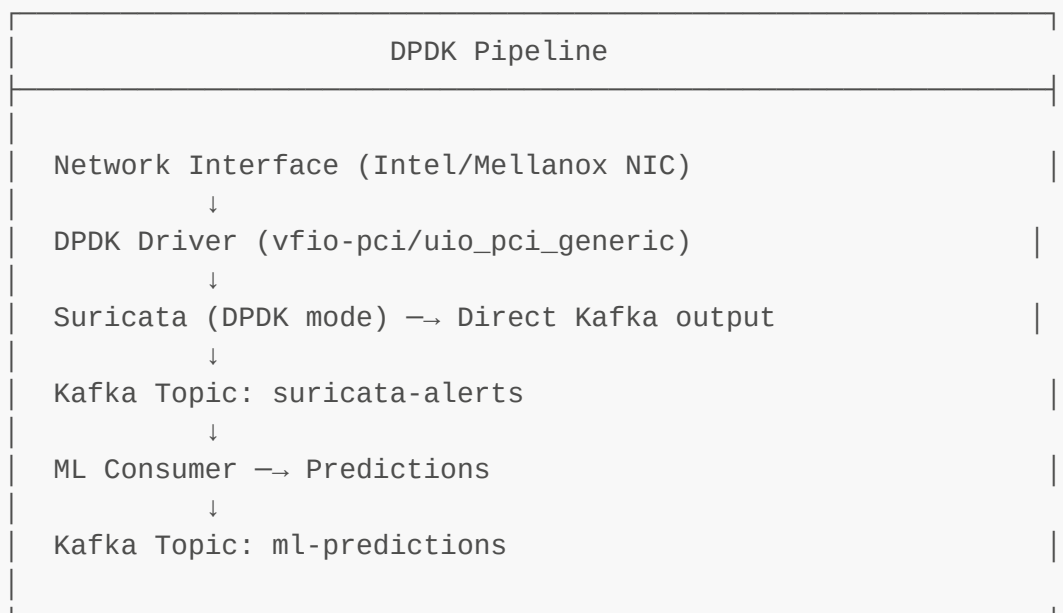
DPDK Mode Runner

What It Does

Purpose: One-stop script to manage the complete IDS pipeline using DPDK mode (high-performance kernel bypass).

File: `run_dpdk_mode.sh`

Architecture It Manages



Key difference from AF_PACKET: No Kafka bridge needed! Suricata writes directly to Kafka.

Additional Functions (DPDK-Specific)

Everything from AF_PACKET mode PLUS these DPDK-specific functions:

1. `check_hugepages()`

What it does:

Verifies and configures Linux hugepages (required for DPDK).

Why hugepages needed:

- DPDK uses large memory pages (2MB instead of 4KB)
- Reduces TLB misses → faster memory access
- Better performance for high-speed packet processing

Implementation:

```
check_hugepages() {
    # Check current hugepages
    hugepages_free=$(cat /sys/kernel/mm/hugepages/hugepages-
2048kB/free_hugepages)
    hugepages_total=$(cat /sys/kernel/mm/hugepages/hugepages-
2048kB/nr_hugepages)

    # Need at least 1024 hugepages (2GB)
    if [ "$hugepages_total" -lt 1024 ]; then
        echo "⚠ Insufficient hugepages (${hugepages_total}/1024)"
        read -p "Configure hugepages now? (y/n): " configure_hp

        if [[ $configure_hp =~ ^[Yy]$ ]]; then
            # Allocate 1024 hugepages
            echo 1024 > /proc/sys/vm/nr_hugepages

            # Mount hugepages filesystem
            mkdir -p /mnt/huge
            mount -t hugetlbfs nodev /mnt/huge

            echo "✓ Hugepages configured"
        fi
    fi
}
```

PROF

What each step does:

1. **Read current hugepages:** Check `/sys/kernel/mm/hugepages/`
 2. **Check if sufficient:** Need $1024 \times 2\text{MB} = 2\text{GB}$
 3. **Prompt user:** Ask before making system changes
 4. **Allocate hugepages:** Write to `/proc/sys/vm/nr_hugepages`
 5. **Mount filesystem:** DPDK needs `/mnt/huge` mount point
-

2. `check_dpdk_binding()`

What it does:

Checks if network interface is bound to DPDK driver.

Why needed:

- DPDK requires interface to use userspace driver
- Interface must be unbound from kernel driver first
- Verifies correct driver is loaded

Implementation:

```
check_dpdk_binding() {
    DEVBIND=$(which dpdk-devbind.py)

    # Auto-detect PCI address if not configured
    if [ -z "$INTERFACE_PCI_ADDRESS" ]; then
        INTERFACE_PCI_ADDRESS=$(ethtool -i "$NETWORK_INTERFACE" | grep
"bus-info" | awk '{print $2}')
    fi

    # Check if bound to DPDK driver
    if $DEVBIND --status | grep "$INTERFACE_PCI_ADDRESS" | grep -q
"drv=$DPDK_DRIVER"; then
        echo "✓ Interface bound to DPDK driver"
        return 0
    else
        echo "⚠ Interface not bound"
        return 1
    fi
}
```

PROF

What it checks:

1. **Find `dpdk-devbind.py`:** DPDK tool for managing drivers
2. **Get PCI address:** Format like `0000:02:00.0`
3. **Check binding status:** Uses `dpdk-devbind.py --status`
4. **Verify driver:** Ensures correct driver (`vfio-pci`/`uio_pci_generic`/`igb_uio`)

Example output from `dpdk-devbind.py --status`:

```
Network devices using DPDK-compatible driver
=====
0000:02:00.0 'Intel 82599ES' drv=vfio-pci unused=ixgbe
```

3. `bind_interface()`

What it does:

Binds network interface from kernel driver to DPDK driver.

Process:

1. Check if already bound (skip if yes)
2. Call `01_bind_interface.sh`
3. Interface goes OFFLINE for normal networking
4. Interface now controlled by DPDK
5. Verify binding succeeded

Implementation:

```
bind_interface() {  
    echo "== Binding Interface to DPDK =="  
  
    if check_dpdk_binding; then  
        return 0 # Already bound  
    fi  
  
    echo "Binding $NETWORK_INTERFACE to DPDK..."  
    bash "${PIPELINE_SCRIPTS}/01_bind_interface.sh"  
  
    if check_dpdk_binding; then  
        echo "✓ Interface bound successfully"  
    else  
        echo "✗ Failed to bind interface"  
        exit 1  
    fi  
}
```

PROF

What happens during binding:

Before:

Interface: ens33
Driver: ixgbe (kernel)
State: UP (usable for SSH, ping)

After:

Interface: 0000:02:00.0
Driver: vfio-pci (DPDK)
State: OFFLINE (only DPDK can use it)

⚠ Warning: You'll lose network access on this interface!

4. `unbind_interface()`

What it does:

Returns interface from DPDK to kernel driver (restores normal networking).

When to use:

- After testing DPDK mode
- Need interface for normal networking again
- Before system shutdown
- Switching back to AF_PACKET mode

Implementation:

```
unbind_interface() {  
    echo "== Unbinding Interface from DPDK =="  
  
    echo "Unbinding $NETWORK_INTERFACE from DPDK..."  
    bash "${PIPELINE_SCRIPTS}/unbind_interface.sh"  
  
    echo "✓ Interface unbound"  
}
```

What happens during unbinding:

```
Before:  
  Interface: 0000:02:00.0  
  Driver: vfio-pci (DPDK)  
  State: OFFLINE  
  
After:  
  Interface: ens33  
  Driver: ixgbe (kernel)  
  State: UP (normal networking restored)
```

PROF

5. `show_dpdn_info()`

What it does:

Displays comprehensive DPDK status information.

Implementation:

```
show_dpdn_info() {  
    echo "== DPDK Information =="
```

```

DEVBIND=$(which dpdk-devbind.py)

if [ -x "$DEVBIND" ]; then
    $DEVBIND --status
else
    echo "x dpdk-devbind.py not found"
fi
}

```

Example output:

```

Network devices using DPDK-compatible driver
=====
0000:02:00.0 'Intel 82599ES 10-Gigabit' drv=vfio-pci unused=ixgbe

Network devices using kernel driver
=====
0000:03:00.0 'Realtek RTL8111' if=eth0 drv=r8169 unused=vfio-pci

Other Network devices
=====
<none>

```

Information shown:

- PCI address (0000:02:00.0)
- Device model (Intel 82599ES)
- Current driver (vfio-pci)
- Available drivers (ixgbe)

DPDK-Specific Startup Sequence

PROF

Modified `start_suricata()` function:

```

start_suricata() {
    echo "== Starting Suricata (DPDK Mode) =="

    if pgrep suricata > /dev/null; then
        echo "⚠ Suricata already running"
        return 0
    fi

    # DPDK-SPECIFIC: Ensure interface is bound
    if ! check_dpdk_binding; then
        echo "Interface not bound to DPDK. Binding now..."
        bind_interface
    fi
}

```

```

bash "${PIPELINE_SCRIPTS}/03_start_suricata.sh"
sleep 3

if pgrep suricata > /dev/null; then
    echo "✓ Suricata started successfully"
else
    echo "✗ Failed to start Suricata"
    exit 1
fi
}

```

Key difference: Automatically binds interface if not already bound.

DPDK-Specific Status Display

Enhanced **show_status()** function:

```

show_status() {
    echo "== System Status =="

    # DPDK binding status
    echo "DPDK Interface Binding:"
    if check_dpdk_binding; then
        echo "✓ Interface bound to DPDK"
        echo "  Interface: $NETWORK_INTERFACE"
        echo "  PCI Address: $INTERFACE_PCI_ADDRESS"
        echo "  Driver: $DPDK_DRIVER"
    else
        echo "✗ Interface not bound to DPDK"
    fi

    # Hugepages status
    hugepages_free=$(cat /sys/kernel/mm/hugepages/hugepages-
2048kB/free_hugepages)
    hugepages_total=$(cat /sys/kernel/mm/hugepages/hugepages-
2048kB/nr_hugepages)
    echo ""
    echo "Hugepages:"
    echo "  Total: $hugepages_total"
    echo "  Free: $hugepages_free"

    # Services status (same as AF_PACKET)
    echo ""
    echo "Services:"
    # ... Kafka, Suricata, ML consumer checks ...
}

```


Additional info shown:

- DPDK binding status
 - PCI address
 - Hugepages allocation
 - Driver in use
-

DPDK-Specific Shutdown

Modified `stop_all()` function:

```
stop_all() {
    echo "== Stopping All Services =="

    # Stop ML consumer
    if pgrep -f "ml_kafka_consumer.py" > /dev/null; then
        pkill -f "ml_kafka_consumer.py"
    fi

    # Stop Kafka bridge (if used)
    if pgrep -f "suricata_kafka_bridge.py" > /dev/null; then
        pkill -f "suricata_kafka_bridge.py"
    fi

    # Stop Suricata
    if pgrep suricata > /dev/null; then
        pkill suricata
        sleep 2
    fi

    # Stop Kafka
    if pgrep -f "kafka.Kafka" > /dev/null; then
        bash "${PIPELINE_SCRIPTS}/stop_all.sh"
    fi

    # DPDK-SPECIFIC: Ask about unbinding interface
    if check_dpdk_binding; then
        echo ""
        read -p "Unbind interface from DPDK? (y/n): " unbind_choice
        if [[ $unbind_choice =~ ^[Yy]$ ]]; then
            unbind_interface
        fi
    fi

    echo "✓ All services stopped"
}
```

Key difference: Prompts user to unbind interface (optional).

Why optional unbinding?

- If restarting soon → keep bound (faster restart)
- If done for the day → unbind (restore normal networking)

DPDK Menu System

Enhanced menu with DPDK options:

```
===== MENU =====
1) Start Complete Pipeline (Kafka + Suricata + ML)
2) Start Kafka Only
3) Start Suricata Only (DPDK)
4) Start ML Consumer Only
5) Start Kafka Bridge Only
6) Bind Interface to DPDK           ← DPDK-specific
7) Unbind Interface from DPDK      ← DPDK-specific
8) Check Status
9) View Logs
10) Show DPDK Info                 ← DPDK-specific
11) Stop All Services
0) Exit
=====
```

Additional options:

- Option 6: Manual interface binding
- Option 7: Manual interface unbinding
- Option 10: View DPDK status info

Common Features

PROF

Both Scripts Share:

1. Error Handling

```
set -e # Exit on error
```

- Any command failure stops script
- Prevents cascading errors
- Makes debugging easier

2. Configuration Loading

```
source "$PIPELINE_CONFIG"
```

- Both scripts load same config file
- Ensures consistent settings
- Single place to change settings

3. Process Management

```
# Check if running
if pgrep -f "process_name" > /dev/null; then
    echo "Already running"
fi

# Kill process
pkill -f "process_name"
```

- Uses **pgrep** to find processes
- Uses **pkill** to stop processes
- **-f** flag matches full command line

4. Dependency Checking

Both validate required software before starting:

- Suricata (with appropriate support)
- Kafka
- Python3
- Additional tools

5. Color-Coded Output

PROF

Both use same color scheme:

- Green = Success
- Red = Error
- Yellow = Warning
- Cyan = Info headers

can you go through this documentation and keep only relevant information in the
DPDK_MODE_ARCHITECTURE.md

I need only high level overview, architecture components which has only relevant brief info, key
functions breakdown, command line usage

put in brief details of how dpdk works and what nics and drivers are compatible with it

refer to AF_PACKET_MODE_ARCHITECTURE.md and keep similar info for DPDK mode as well.

make changes only in the DPDK_MODE_ARCHITECTURE.md and not any other file

6. Interactive & Non-Interactive Modes

Both support:

- Menu-driven interface (no arguments)
- Command-line operation (with arguments)

Usage Examples

AF_PACKET Mode

Interactive Mode

```
sudo ./run_afpacket_mode.sh

# Menu appears, select option:
Enter choice [0-10]: 1    # Start complete pipeline
```

Command-Line Mode

```
# Start everything
sudo ./run_afpacket_mode.sh start

# Individual components
sudo ./run_afpacket_mode.sh kafka
sudo ./run_afpacket_mode.sh suricata
sudo ./run_afpacket_mode.sh bridge
sudo ./run_afpacket_mode.sh ml

# Check status
sudo ./run_afpacket_mode.sh status

# Stop everything
sudo ./run_afpacket_mode.sh stop
```

Typical Workflow

```
# 1. First time setup
sudo ./run_afpacket_mode.sh setup    # Configure interface

# 2. Start pipeline
sudo ./run_afpacket_mode.sh start    # Start all components
```

```
# 3. Check everything is running
sudo ./run_afpacket_mode.sh status

# 4. View logs to verify
sudo ./run_afpacket_mode.sh logs

# 5. When done, stop everything
sudo ./run_afpacket_mode.sh stop
```

DPDK Mode

Interactive Mode

```
sudo ./run_dpdk_mode.sh

# Menu appears, select options:
Enter choice [0-11]: 6      # Bind interface first
Enter choice [0-11]: 1      # Start complete pipeline
```

Command-Line Mode

```
# Bind interface first (one-time)
sudo ./run_dpdk_mode.sh bind

# Start everything
sudo ./run_dpdk_mode.sh start

# Check DPDK info
sudo ./run_dpdk_mode.sh info

# Check status
sudo ./run_dpdk_mode.sh status

# Stop and unbind
sudo ./run_dpdk_mode.sh stop
# (Will prompt to unbind)
```

Typical Workflow

```
# 1. One-time DPDK setup
sudo ./run_dpdk_mode.sh bind      # Bind interface to DPDK

# 2. Start pipeline
```

```

sudo ./run_dpdk_mode.sh start          # Start all components

# 3. Verify DPDK status
sudo ./run_dpdk_mode.sh info          # Show DPDK binding info

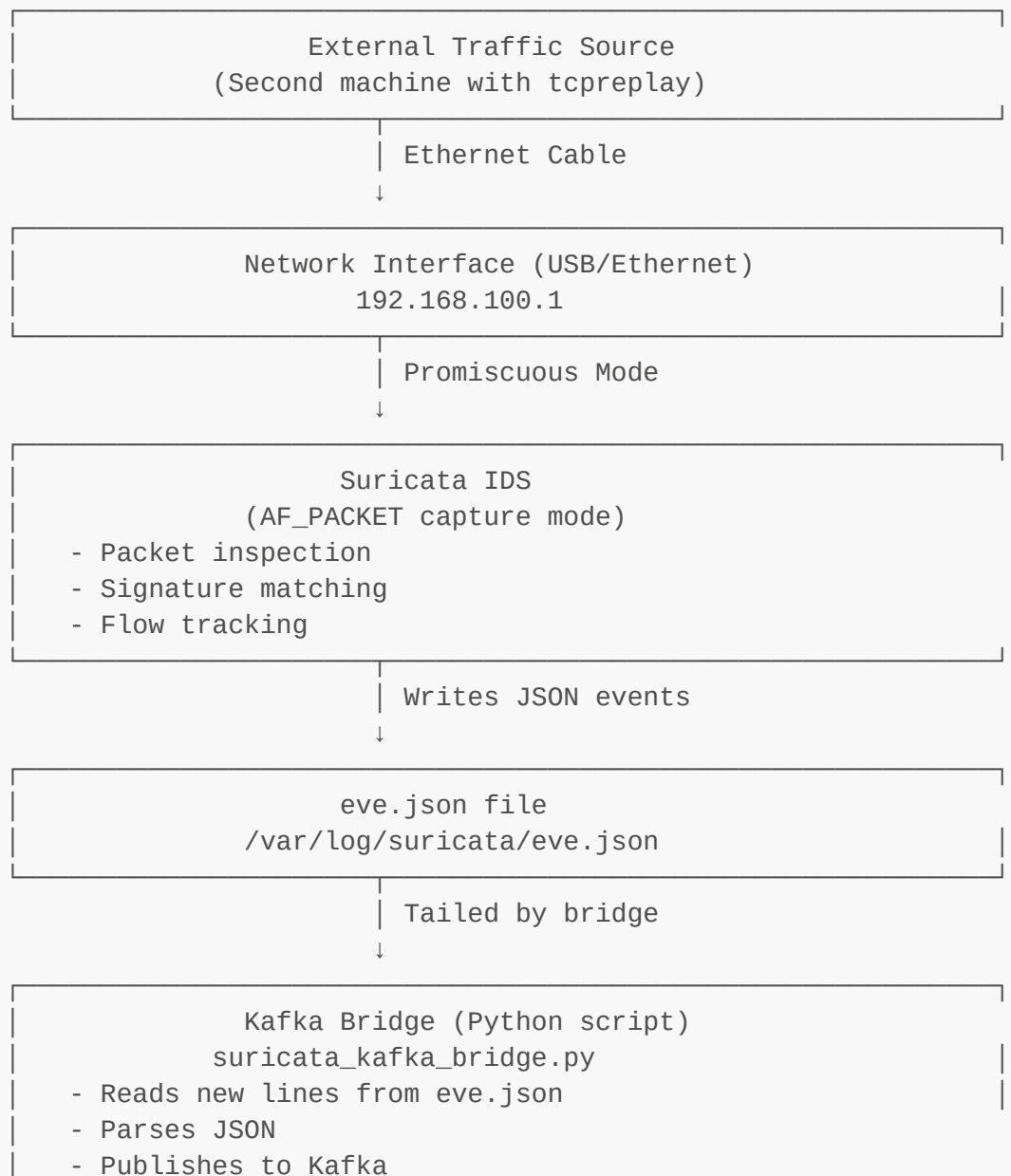
# 4. Check everything running
sudo ./run_dpdk_mode.sh status

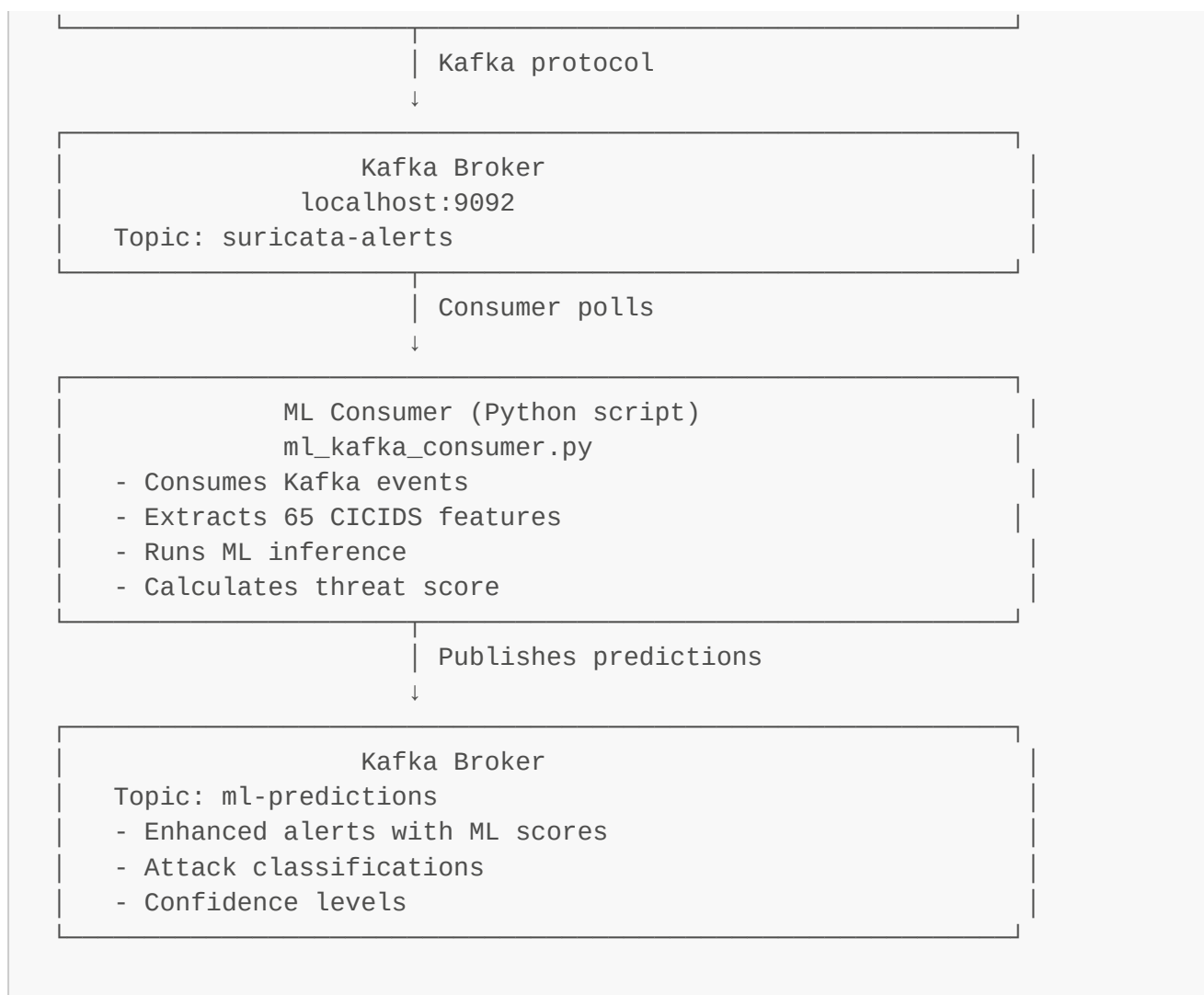
# 5. When done, stop and restore interface
sudo ./run_dpdk_mode.sh stop
# Answer 'y' when prompted to unbind

```

Architecture Diagrams

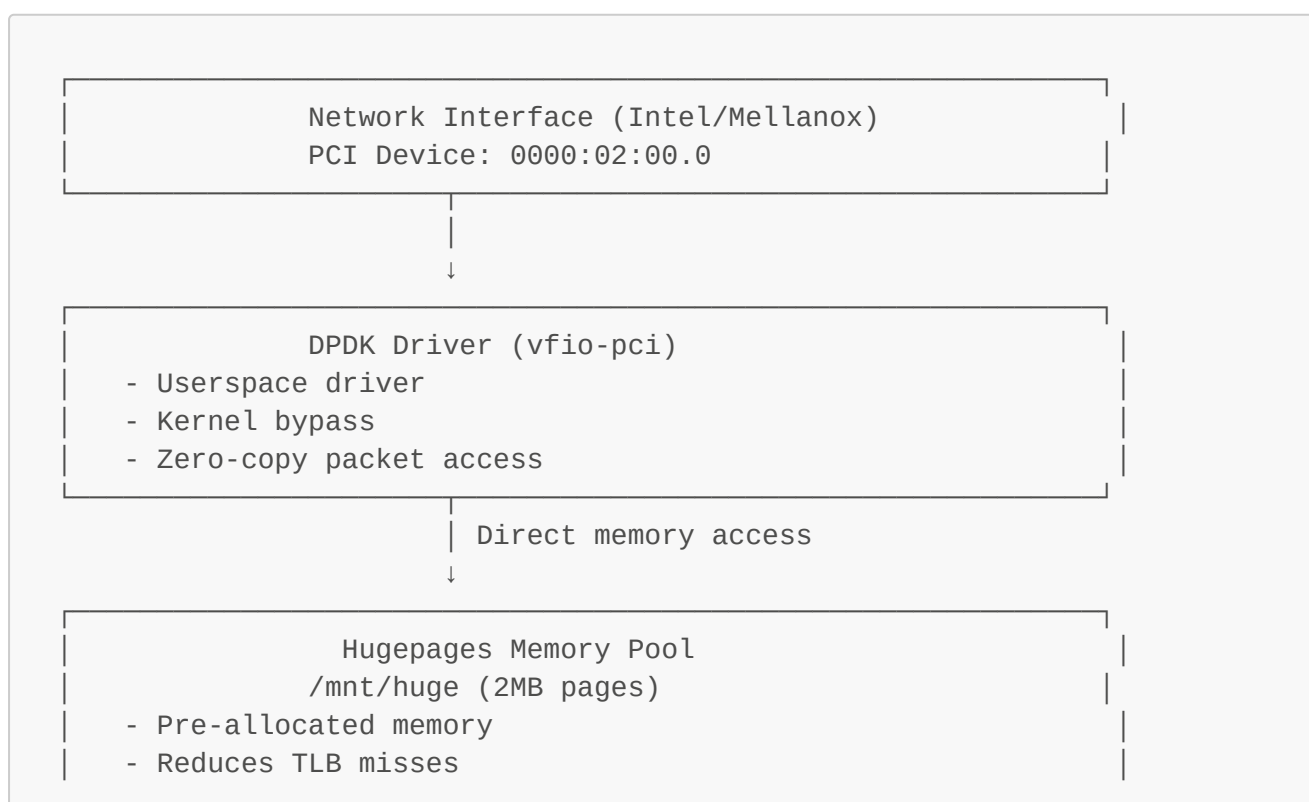
AF_PACKET Mode Data Flow

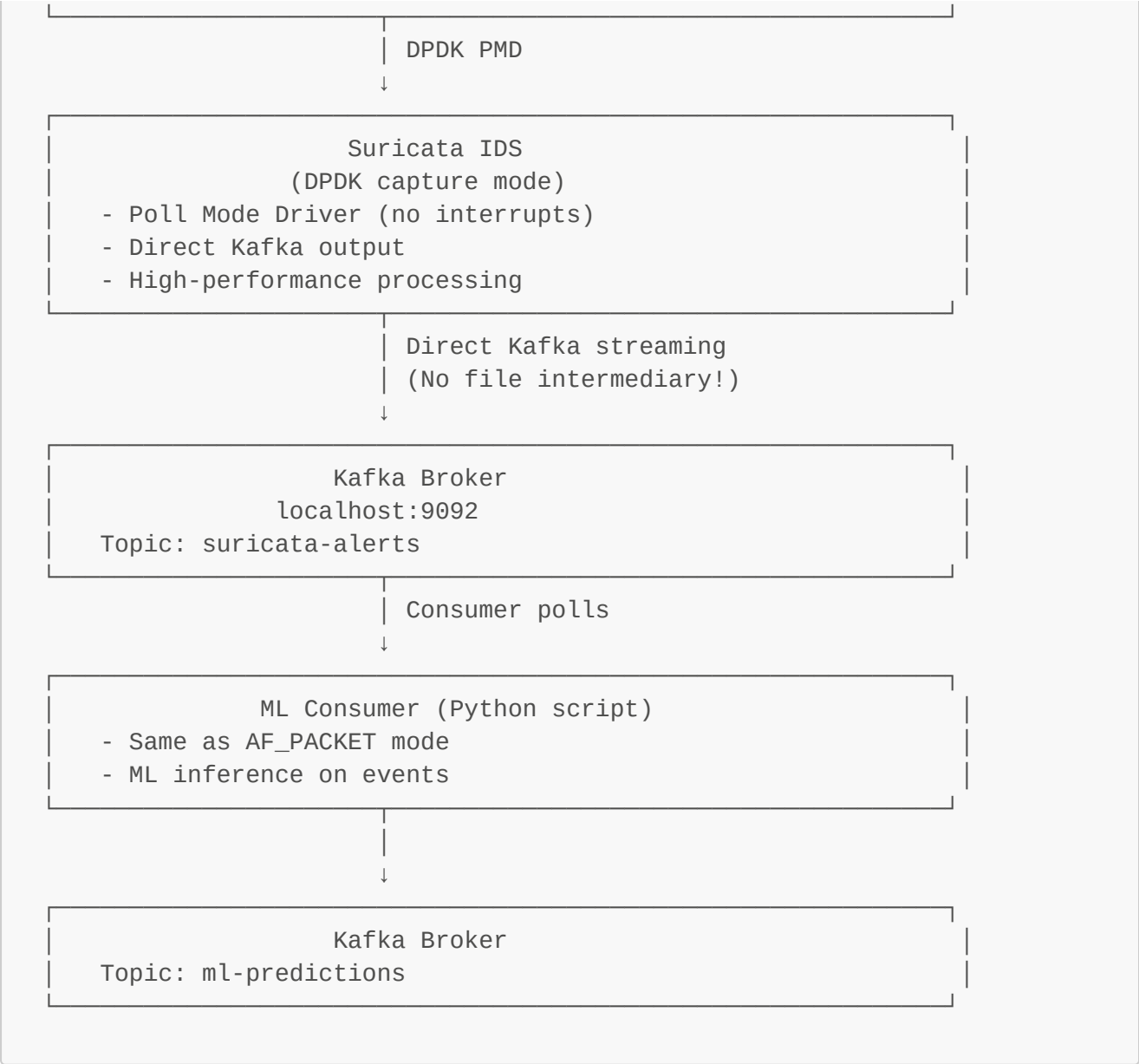




DPDK Mode Data Flow

PROF





Key differences:

- No Kafka bridge needed (direct Kafka output)
- Kernel bypass for performance
- Hugepages for efficient memory
- Poll Mode Driver (no interrupts)

Script Comparison

Feature	AF_PACKET Mode	DPDK Mode
Interface Binding	Not needed	Required (<code>bind_interface()</code>)
Hugepages	Not needed	Required (<code>check_hugepages()</code>)
Kafka Bridge	Required	Not needed
Interface State	UP (usable)	OFFLINE (dedicated)

Feature	AF_PACKET Mode	DPDK Mode
Throughput	100-500 Mbps	1-10+ Gbps
Setup Complexity	Simple	Complex
Compatible NICs	ALL (USB, Ethernet, WiFi)	Intel, Mellanox, Broadcom only
Root Required	Yes (Suricata)	Yes (DPDK + Suricata)
Menu Options	10 options	11 options
Dependencies	Suricata, Kafka, Python	Same + DPDK tools

Automation Examples

Using in Systemd Service

AF_PACKET Mode:

```
[Unit]
Description=IDS Pipeline (AF_PACKET Mode)
After=network.target

[Service]
Type=forking
ExecStart=/path/to/run_afpacket_mode.sh start
ExecStop=/path/to/run_afpacket_mode.sh stop
Restart=on-failure
User=root

[Install]
WantedBy=multi-user.target
```

PROF

DPDK Mode:

```
[Unit]
Description=IDS Pipeline (DPDK Mode)
After=network.target

[Service]
Type=forking
ExecStartPre=/path/to/run_dpdk_mode.sh bind
ExecStart=/path/to/run_dpdk_mode.sh start
ExecStop=/path/to/run_dpdk_mode.sh stop
Restart=on-failure
User=root

[Install]
WantedBy=multi-user.target
```

Using in Cron Job

Start at boot:

```
@reboot /path/to/run_afpacket_mode.sh start >> /var/log/ids_startup.log 2>&1
```

Daily status check:

```
0 9 * * * /path/to/run_afpacket_mode.sh status | mail -s "IDS Status" admin@example.com
```

Shell Script Integration

```
#!/bin/bash
# deploy_ids.sh - Deploy and start IDS

# Choose mode based on interface type
INTERFACE_TYPE="usb" # or "pcie"

if [ "$INTERFACE_TYPE" == "usb" ]; then
    echo "Deploying AF_PACKET mode..."
    sudo ./run_afpacket_mode.sh setup
    sudo ./run_afpacket_mode.sh start
else
    echo "Deploying DPDK mode..."
    sudo ./run_dpdk_mode.sh bind
    sudo ./run_dpdk_mode.sh start
fi

# Wait and check status
sleep 10
sudo ./run_${INTERFACE_TYPE}_mode.sh status
```

PROF

Troubleshooting

Common Issues

"Must be run as root"

Cause: Script requires sudo

Solution:

```
sudo ./run_afpacket_mode.sh
```

"Configuration file not found"

Cause: Config file missing

Solution:

```
# Check if file exists
ls dpdk_suricata_ml_pipeline/config/pipeline.conf

# If missing, copy from template
cp pipeline.conf.template pipeline.conf
```

"Interface not found"

Cause: Wrong interface name in config

Solution:

```
# List available interfaces
ip link show

# Update config file
nano dpdk_suricata_ml_pipeline/config/pipeline.conf
# Set NETWORK_INTERFACE="correct_name"
```

PROF

"Failed to start Kafka"

Cause: Kafka already running or port 9092 in use

Solution:

```
# Check what's using port 9092
sudo netstat -tulpn | grep 9092

# Kill existing Kafka
pkill -f kafka

# Restart
sudo ./run_afpacket_mode.sh kafka
```

DPDK-Specific: "Insufficient hugepages"

Cause: Not enough hugepages allocated

Solution: Script will prompt to configure automatically, or:

```
# Manual configuration
echo 1024 > /proc/sys/vm/nr_hugepages
mkdir -p /mnt/huge
mount -t hugetlbfs nodev /mnt/huge
```

DPDK-Specific: "Failed to bind interface"

Cause: Interface in use or wrong driver

Solution:

```
# Check current status
dpdk-devbind.py --status

# Ensure interface is down first
sudo ip link set ens33 down

# Try binding again
sudo ./run_dpdk_mode.sh bind
```

Best Practices

1. Always Check Status After Starting

```
sudo ./run_afpacket_mode.sh start
sudo ./run_afpacket_mode.sh status
```

2. Use Command-Line Mode for Automation

```
# Not for automation (requires interaction):
sudo ./run_afpacket_mode.sh

# Good for automation (non-interactive):
sudo ./run_afpacket_mode.sh start
```

3. Stop Properly Before System Shutdown

```
# Ensures clean shutdown of all components
sudo ./run_afpacket_mode.sh stop
```

4. For DPDK: Always Unbind Before Maintenance

```
sudo ./run_dpdk_mode.sh stop
# Answer 'y' to unbind prompt
```

5. Monitor Logs Regularly

```
# Quick log check
sudo ./run_afpacket_mode.sh logs

# Or use monitoring script
./monitor_traffic.sh
```

Summary

What These Scripts Are:

Master control scripts that orchestrate the entire IDS pipeline - they're your one-stop shop for:

-  Starting all components in correct order
-  Checking if everything is running
-  Viewing logs
-  Stopping gracefully
-  Managing DPDK binding (DPDK mode)

Why They're Useful:

Instead of running 5+ individual scripts manually, these runners:

- Handle dependencies automatically
- Verify each component started successfully
- Provide visual feedback with colors
- Offer both interactive menu and command-line modes
- Simplify complex operations (DPDK binding, hugepages)

When to Use Each:

Use **run_afpacket_mode.sh** when:

- Using USB Ethernet adapter

- Testing/development
- Any network interface
- Want simplicity

Use `run_dpdk_mode.sh` when:

- Using Intel/Mellanox/Broadcom NIC
- Need maximum performance (> 1 Gbps)
- Production high-traffic environment
- Have DPDK-compatible hardware

End of Documentation

Pro Tip: These scripts are your "Easy Button" for the IDS pipeline. Master these, and you can start/stop/manage the entire system with a single command!